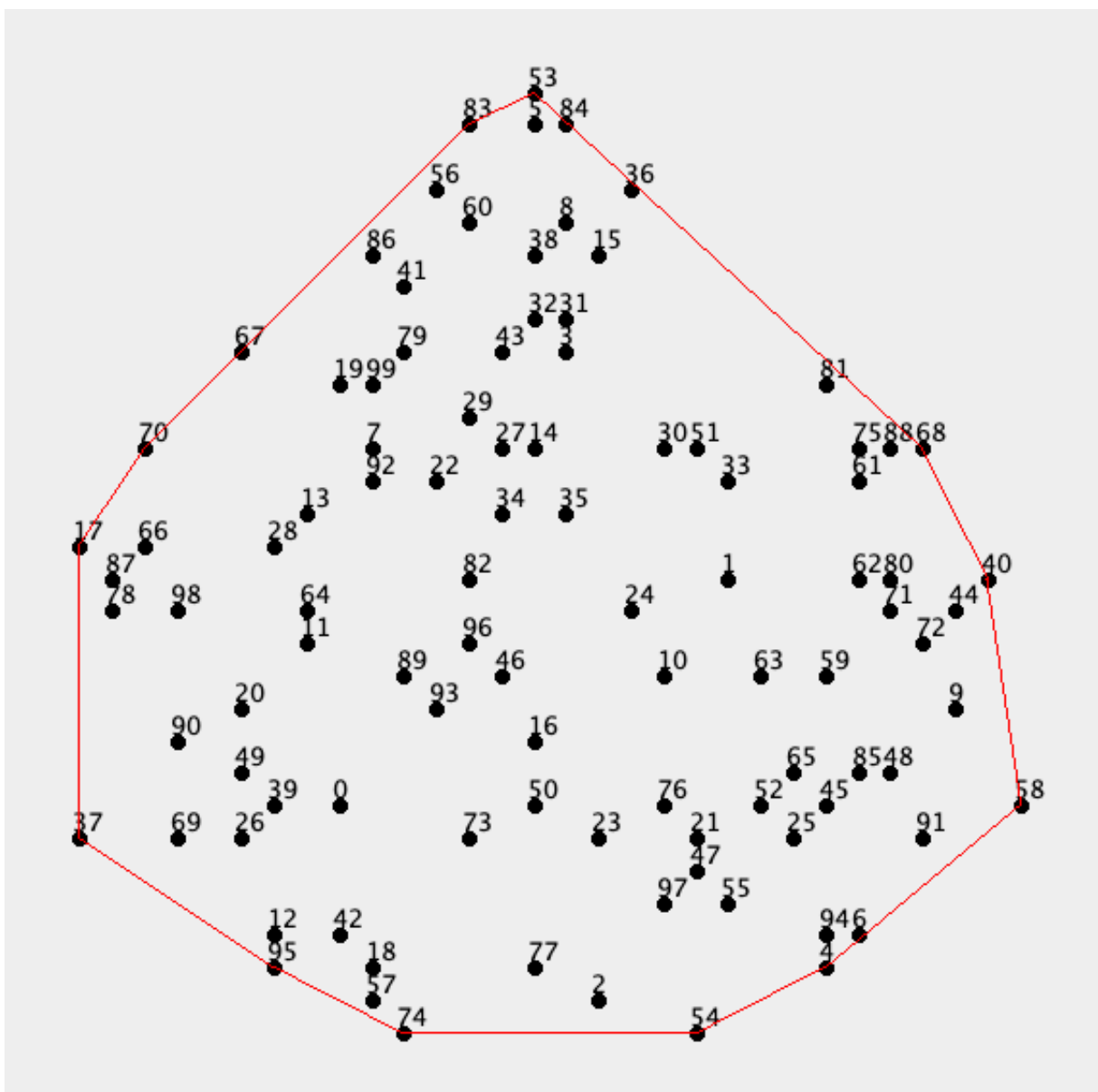


Report IN3030 – Oblig 4

Processor:

- Apple M1-chip, 3228 MHz
- 8 core cpu

The task given was to find the convex hull of a set of points. To do we had to implement the algorithm specified in the oblig text. The algorithm used in this oblig is a recursive one where you first start draw a straight line from the point with the lowest x-value, to the one with the highest. Afterwards we find the point furthest away from this line and split it up. This is repeated in a recursive way until there are no points outside the circle created. It will look like this. For this model I used "java Main 100 3333 8". The number of threads does not influence the model.



How to use the program:

First compile the files using “javac *.java” in the terminal. The file that has the main program is Main.java, so to run the program you use:

Main <number of points> <seed> <number of threads>

Example: Main 1000 33 4

Sequential Implementaion

For the sequential part I did not do any changes to the given precode other than give the ConvexHull class a few local variables. I gave it “points” which is of the type NPunkter17. This is because used the NPunkter17 class in the sequential version to make a new IntList for the recursive algorithm to fill. I also gave it an IntList “hull” which will be the return value from running the sequential program.

The sequential class inherits from the ConvexHull. I did it this way mostly bcause the Oblig4Precode constructor asked for a convex hull. So it made it easier, but also because it had a lot of values that I would need. So instead of doing function calls I could just call upon the local variables.

Before the recursive calls start we first fill the arrays using the fyllArrayer given in the precode. Then we determine MIN/MAX X/Y which is the index to the biggest/smallest point. After that we add MIN_X to “hull” and call upon the recursive method.

In the recursive method we use a for-loop to iterate through the valid points. We determine the valid points by creating a IntList using the NPunkter17s lagIntList() method. D is set to 0 at first but in the for-loop it will hold a value which is the product of

$a * x[\text{validpoints.get}(i)] + b * y[\text{validpoints.get}(i)] + c$

It then checks to see if D is smaller than 0 and keeps track of the lowest point we have come across. For every points that is lower than the previous, we add it to a list to possible points and keep track of its index in our validpoints list.

After the for-loop is done, we should have found the lowest point and also found possible points for the next recursive call. We add the lowest point to our hull then continue in our recursive call. If we the lowest haven't been found (if d is greater than 0) we come to the base case where the recursive call stops. Here we sort through all the values for d when it was 0. Here we sort through all the values that did not get added in the for-loop and we use the sorting algorithm “sort” to make sure they are sorted in the right order. We append the list to our hull after that. We come back to the “calcHull”

method and return our local variable “hull”. We repeat this 7 times and return the median time it took for the program to complete the sequential version.

Parallel Implementaion

Parallel also makes use of the ConvexHull Class as it also extends the class. The parallel class makes use of the thread amount the user specified during the programs launch. The start of the parallel program is similar to the sequential, since it also initiates the local variable “hull” for the convexhull and also fills an IntList consisting of all the number $x \leq n$ to be used to see if the points are valid or not.

It start of by calling the “calcHull” method that the parallel has. (note. Could probably make the method abstract in the convexHull-class and override it in the different classes.) It does what we have already said above and starts the recursion with first one thread with the for the values $\text{MIN_X} \rightarrow \text{MAX_X}$, and a second thread for $\text{MAX_X} \rightarrow \text{MIN_X}$.

The thread class takes: ConvexHull parent_p, int thread_amount, int idx_start, int idx_stop, IntList validpoints, IntList hullpoints as arguments.

Parent_p is a ConvexHull, but in this case the Parallel, and we will be using its x[] and y[] arrays that we made using the method fyllArrayer() in this and the deeper recursion levels to define “d”.

The thread_amount is used as a measurement and to determine the base case for the recursion calls. The thread amount will decrease the deeper the calls go and we will stop making new threads.

The start and stop are used to calculate the hull. In the first recursive call start and stop will be MIN/MAX_X, but it will change as start will be the lowest point found. This is stored in d_idx. And this repeats itself.

Validpoints is an IntList consisting of all the nebers from 0 to n and hullpoints is our Intlist where we will add and append our results from the threads.

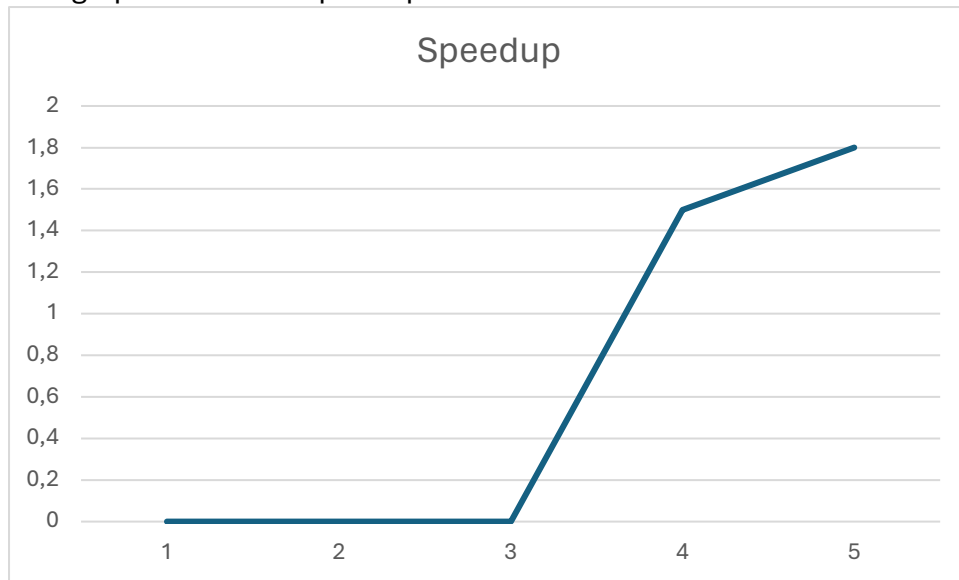
The initiation of our threads can be look at as a tree where we first start with one, and then it branches out. When the thread amount is 1 or 0 it will return the result up to the top layer. On the way to the top it will append its results with the previous recursive call where it was from.

Performance

When I took the median time, they performed the same on both $n = 1000$, 10.000 and 100.000 , but when I took a closer look on 10.000 and 100.000 sequential had a better average. I think this might come from the behaveior of the threads and how they run together.

N	Sequential ms	Parallel ms	Speedup
1000	0	0	0
10000	0	0	0
100000	4,194	4,194	0
1000000	25.126	16.777	1.5
10000000	268.435	146.800	1.800

This graph shows the speedup.



Conclusion

We can see that there were some speedup when n increased, but on closer inspection the sequential had a better average except on $n = 10.000.000$. This might be because of the workload is not necessarily distributed properly where one thread might finish earlier than the others. A way we can look at this is by looking at the recursive calls as a tree, and since it is not balanced some threads might not do as much work as others. Even if we used 8 threads, the speedup is not 1 to 1. This is also because the algorithm makes use of a recursive function which makes it impossible to make it 1 to 1.

We see that the biggest speed up is from 10.000 to 100.000 but the speedup stagnates. This might be due to distribution of the workload between the threads as mentioned earlier. When I tried other seeds I got different results, which is also worth mentioning. (I used seed = 33 and 8 threads)